EDUIGNITE

OPERATING SYSTEM

The Uploadable App Platform

Architecture — for review
Apps built separately, uploaded, reviewed and updated
without touching EduIgnite.

Powered by EduIgnite · Bamenda, Cameroon

1. What we are building

Today an EduIgnite app is a Django module compiled into the release. Adding one means a backend deploy plus new mobile and desktop builds.

We want apps that are **built separately, uploaded, reviewed, published and updated on their own schedule, without touching EduIgnite itself**. A school installs one from the store and it appears in their account.

The model is VS Code: apps ship real code, run inside a host the platform controls, and contribute UI that the platform draws.

The four decisions this rests on

These were settled in discussion. Everything in this document follows from them.

Decision	Consequence
App code runs inside the EduIgnite runtime	We own the execution host; apps never run loose on the server.
App data lives in the school database	The platform provides storage; apps never create tables or run schema migrations.
Apps contribute UI, the runtime draws it	Apps ship UI code, but inside a frame we control — never their own window or site.
Only our own team writes apps	The sandbox contains our bugs, not an attacker. A far easier problem.

What an app is

A versioned package containing server logic, UI code, a declared data shape and a declared set of permissions. It is not a website, not an APK and not a Django app. It cannot exist outside the runtime.

2. The two lifecycles

Conflating these is the usual source of confusion, so they are kept separate.

Store side — what a build is doing globally

```
Draft -> Submitted -> Automated checks -> In review -> Approved -> Published
                            |                  |
                        Rejected --------------+   (back to Draft)

Published -> Deprecated -> Pulled
```

School side — what one school is actually running

```
Available -> Installed (pinned to version X) -> Update available -> Updated
                    |
               Uninstalled (data retained)
```

A school runs v1 while v2 is published. Both are true at once, and both are stored. Uninstalling never deletes the school's records — the same rule the existing registry already follows.

3. Package format

An `.eia` file is a zip:

```
manifest.json     what the app is, what it needs, what it contributes
server/           Python package - app logic
web/              built JS bundle(s) - app UI
assets/           icons, images
migrations/       declared data migrations between app versions
CHANGELOG.md      shown to schools when an update is offered
```

Nothing outside `server/` and `web/` executes. Assets are served as static files. There is no install script and no hook that runs at upload time.

4. Manifest

```
{
  "key": "teacher_attendance",     // permanent; installations reference it
  "version": "1.2.0",              // semver; every submission is a version
  "runtime_api": "1.x",            // host API range this build needs
  "name": "Teacher Attendance",
  "summary": "Clock-in register and monthly reports for teaching staff.",
  "icon": "ClipboardCheck",
  "publisher": "EduIgnite",

  "scope": "both",                 // school | user | both
  "pricing": { "model": "subscription", "price": 15000, "currency": "XAF" },
  "roles": ["SCHOOL_ADMIN", "SUB_ADMIN", "TEACHER"],

  "collections": [ ... ],
  "screens":     [ ... ],
  "permissions": { ... },
  "jobs":        [ ... ]
}
```

> **key is permanent.** Changing it orphans every installation and every row of school data, so the review pipeline rejects a submission whose key does not match the published app.

`scope` and `pricing` already exist in the shipped registry and mean the same things. `both` is “the school buys the licence and each user opts in”, which is exactly the Teacher Attendance case.

5. Data

Apps **declare collections**. They never create tables and never run schema migrations against the EduIgnite database. This is the single most important constraint in the design: an uploaded package that could alter the schema holding student records is not something we can make safe.

```
"collections": [
  {
    "name": "clock_events",
    "fields": [
      { "name": "teacher", "type": "reference", "to": "core.user",
        "required": true, "indexed": true },
      { "name": "date",    "type": "date", "required": true, "indexed": true },
      { "name": "status",  "type": "enum", "values": ["present","absent","late"] },
      { "name": "minutes_late", "type": "integer" }
    ]
  }
]
```

Storage

One platform-owned table holds every app's records:

```
app_records(app_key, school_id, collection, record_id, data JSONB, created, modified)
```

Fields marked `indexed` get GIN/expression indexes so real queries stay fast. If a collection outgrows JSONB — high volume, heavily queried — that is a signal it should be a built-in app instead, and the review gate is where we notice.

Isolation

This is the real security boundary, not the language sandbox. Apps get no ORM, no SQL and no connection. They get an SDK, and the SDK injects `app_key` and `school_id` into every read and write. An app *cannot express* a query reaching another app's collections or another school's rows, because there is no API surface through which to say it.

A language-level sandbox can be escaped. A missing API cannot be called.

6. Permissions

An app declares everything it touches. The runtime enforces the declaration, and review checks the declaration against what the code actually does.

```
"permissions": {
  "core":        ["students:read", "users:read", "classes:read"],
  "collections": ["clock_events:read", "clock_events:write"],
  "notify":      ["staff"],
  "jobs":        true
}
```

Core access is read-only in v1. An app that could write student records or marks is one bad build away from corrupting a term of data, and nothing in the first four apps we want needs it. Revisit when something real requires it.

Core reads go through the existing service boundaries — `apps.students.api` and the ones that follow it — so apps are subject to the same rules as first-party code, and the boundary guard keeps applying.

7. UI

Apps ship real UI code. It is loaded into an **iframe inside the EduIgnite dashboard chrome** — the approach VS Code uses for webviews.

```
"screens": [
  { "id": "register", "title": "Daily Register", "route": "/register",
    "roles": ["SCHOOL_ADMIN", "TEACHER"], "entry": "web/register.js" },
  { "id": "report", "title": "Monthly Report", "route": "/report",
    "roles": ["SCHOOL_ADMIN"], "entry": "web/report.js" }
]
```

The platform draws the navigation entry, the page frame, the title and the breadcrumbs. The app draws inside the panel.

Why an iframe

App UI **never receives the auth token and never touches the host DOM**. It asks the bridge for data and gets back only what its declared permissions allow. Even with trusted authorship this is worth having: a mistake in an app cannot leak a session, scrape a page, or interfere with a screen it does not own.

The bridge, over postMessage:

```
bridge.data.query('clock_events', { where: { date: today }, limit: 200 })
bridge.data.insert('clock_events', { teacher, date, status })
bridge.core.students.list({ school_class })      // read-only
bridge.call('server_handler_name', args)          // invoke app server logic
bridge.ui.toast('Saved') / bridge.ui.confirm(...)
bridge.theme                                      // tokens, so apps look native
```

8. Execution

App server logic runs in a **dedicated host process**, never in gunicorn.

```
client -> EduIgnite API -> permission check -> RPC -> app host process -> SDK -> database
```

Per call: a wall-clock timeout, a memory cap, no filesystem, no outbound network. A crash or a runaway loop takes down one app host worker; a supervisor restarts it and the platform never notices.

Scheduled work is declared, not self-scheduled:

```
"jobs": [ { "name": "monthly_report", "schedule": "0 6 1 * *",
            "handler": "jobs.monthly_report" } ]
```

On the word “sandbox”

> In-process Python sandboxing is **not** a security boundary. RestrictedPython, `exec` with stripped globals and similar approaches have documented escapes, and we should not pretend otherwise in any document a school might read.

What actually protects the platform, in order of importance:

- apps reach data only through a scoped SDK (§5);
- logic runs in a separate process with limits (§8);
- nothing runs until a human has reviewed and approved it (§10);
- only our own team can publish.

Given trusted authorship, that combination is sufficient. It contains bugs, which is the actual threat. It would not stop a determined insider, and we should not claim it does.

9. Host API versioning — the permanent cost

Once apps ship code against the runtime API, **we cannot break it without breaking apps**. Every exposed function is a long-term commitment.

- the manifest declares `runtime_api`;
- the platform advertises the range it supports;
- an incompatible app is **refused with a clear message, not loaded and crashed**;
- breaking changes require a major version, and the previous major stays supported until every published app has moved.

Cheap to build now. Impossible to retrofit once apps exist in the wild.

10. Review pipeline

Automated checks first, then a person:

- manifest validates against schema; `key` matches the published app; version increments;
- declared permissions match what static analysis finds the code doing;
- no forbidden imports (`socket`, `subprocess`, `os`, raw DB access);
- migrations declare a reverse;
- bundle size within budget — these clients run on mid-range Android over slow connections;
- **the build runs against a staging school with test data and must pass.**

That last one catches the class of mistake self-review reliably misses.

> **Honest limitation.** Our team writes the apps and our team reviews them. That is a quality gate — regressions, bad migrations, slow queries — not an adversarial one. It should never be described to schools as though it verifies intent.
>
> If review is to have teeth, the author and the approver should be different people. Worth deciding as policy rather than leaving to habit.

11. Updates and rollback

Schools **pin to a version**. Updates are *offered*, not pushed:

- a published update shows in the school's app list with its changelog;
- an administrator chooses when to take it;
- founders can **force** a build when the current one is actively harmful — the “halt rollout” equivalent;
- rollback re-points a school at the previous approved version.

Silent auto-update is the wrong default here. A school mid-term cannot stop taking attendance while a surprise update settles.

Data migrations between app versions are declared in `migrations/`, run on update, and must declare a reverse so a rollback is not a data loss event.

12. Offline

Non-negotiable — these schools lose power and network routinely.

- manifest and UI bundles cached on device, keyed by app version;
- collection data cached per app; writes queued and synced on reconnect;
- the pinned version means a device offline for a week comes back to the version it left, not to a surprise.

13. Explicitly out of scope for v1

Named so nobody has to guess.

Excluded	Why
Third-party publishers	Only our team publishes. Removes the hostile-code problem entirely.
Apps writing Core data	Read-only until something real needs otherwise.
Outbound network calls from apps	No app reaches the internet from our server.
Payment apps	A wallet accepting online money needs gateway credentials, a ledger, reconciliation, refunds and an audit tra
New UI component types	The component vocabulary belongs to the platform.

14. Build sequence

Each stage is useful on its own.

#	Stage	Produces
1	Package, upload, review, versioning	Apps exist and can be approved. Nothing executes yet.
2	Data service and SDK	School and app scoping enforced. Testable without any app.
3	Server execution host	Process isolation, limits, RPC.
4	Client bridge and iframe renderer	Web first.
5	Desktop and mobile	Plus offline bundle caching.
6	Teacher Attendance as the first real app	Proves all of it end to end.

Teacher Attendance is deliberately the proving app rather than a content app like GCE Past Questions. It writes data, has roles, needs a report and a scheduled job. A content app would exercise almost none of the runtime and would teach us the wrong lessons.

15. Open decisions

Not settled. Each one changes the build.

#	Decision	What hangs on it
1	Python or JavaScript for app logic?	Python fits the existing stack and team. JavaScript would mean one language for both halves of
2	Will apps ever write Core data?	Read-only is the v1 position. If Teacher Attendance should update a teacher record, the permiss
3	Update default — offered or forced?	And who may force one.
4	Author ≠ approver?	Decides whether review is a real gate.
5	Data volume ceiling	At what point does a collection stop being JSONB and become a signal that the app should be b

16. What this costs

This is the largest piece of work in the project by a wide margin: a package format, an upload and review pipeline, versioning, a process-isolated execution host, a scoped data SDK, a message bridge, an iframe renderer in three clients and offline bundle caching — before a single app exists.

The App Registry already shipped is the foundation it sits on: installations, scopes, pricing, licence expiry and the console are built and working. An uploaded app is an installation like any other.

> Worth being clear-eyed that stages 1–5 produce **no visible feature for a school**. The first thing a school sees is stage 6.

This document is a design for review. Nothing described here is built.